## loading the data & merging

```
! gdown 1NyM9EFl7yESFRV6euGteOun6B3ATpPrY
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1NyM9EFl7yESFRV6euGteOun6B3ATpPrY
From (redirected): https://drive.google.com/uc?id=1NyM9EFl7yESFRV6euGteOun6B3ATpPrY&confirm=t&uuid=deb71055-c8d0-488a-9f80-c
To: /content/olist.sqlite
100% 113M/113M [00:03<00:00, 31.3MB/s]
```

```
import sqlite3
import pandas as pd

# connect to sqlite file
conn = sqlite3.connect("olist.sqlite")

# your table list
tables = [
    "product_category_name_translation",
    "sellers",
    "customers",
    "geolocation",
    "order_items",
    "order_payments",
    "order_reviews",
    "orders",
    "products",
    "leads_qualified",
    "leads_closed"
]

# loop through tables and export each to CSV
for table in tables:
    df = pd.read_sql(f"SELECT * FROM {table}", conn)
    df.to_csv(f"{table}.csv", index=False)
    print(f"✅ Exported: {table}.csv")

# close connection
conn.close()
```

```
✅ Exported: product_category_name_translation.csv
✅ Exported: sellers.csv
✅ Exported: customers.csv
✅ Exported: geolocation.csv
✅ Exported: order_items.csv
✅ Exported: order_payments.csv
✅ Exported: order_reviews.csv
✅ Exported: orders.csv
✅ Exported: products.csv
✅ Exported: leads_qualified.csv
✅ Exported: leads_closed.csv
```

```
import pandas as pd

def load_data():
    customers = pd.read_csv(f"customers.csv")
    orders = pd.read_csv(f"orders.csv")
    payments = pd.read_csv(f"order_payments.csv")
    reviews = pd.read_csv(f"order_reviews.csv")

    return customers, orders, payments, reviews


def merge_data():
    customers, orders, payments, reviews = load_data()

    # -------------------------
    # Step 1: customers + orders
    # -------------------------
    df = orders.merge(customers, on="customer_id", how="left")

    # -------------------------
    # Step 2: + payments
    # -------------------------
    df = df.merge(payments, on="order_id", how="left")

    # -------------------------
    # Step 3: + reviews
```

```
    # -------------------------
    df = df.merge(reviews, on="order_id", how="left")

    return df
```

```
pd.set_option('display.max_columns', None)
```

```
df = merge_data()
df.head()
```

| | order_id | customer_id | order_status | order_purchase_timestamp | order_approve |
|---|---|---|---|---|---|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:0 |
| 1 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:0 |
| 2 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:0 |
| 3 | 53cdb2fc8bc7dce0b6741e2150273451 | b0830fb4747a6c6d20dea0b8c802d7ef | delivered | 2018-07-24 20:41:37 | 2018-07-26 03:2 |
| 4 | 47770eb9100c2d0c44946d9cf07ec65d | 41ce2a54c0b03bf3443c3d931a367089 | delivered | 2018-08-08 08:38:49 | 2018-08-08 08:5 |

```
df.shape
```

```
(104478, 22)
```

```
print(f"total id's: {df['customer_id'].nunique()} & null_count: {df['customer_id'].isna().sum()}")
print(f"total id's: {df['customer_unique_id'].nunique()} & null_count: {df['customer_unique_id'].isna().sum()}")
#clearly, some of customer_id's are assined multiple for a single customer, so we can drop customer_id
```

```
total id's: 99441 & null_count: 0
total id's: 96096 & null_count: 0
```

```
df = df.drop(columns=[
    'customer_id',
    'review_id',
    'review_comment_title',
    'review_comment_message',
    'review_creation_date',
    'review_answer_timestamp'
], errors='ignore')
```

```
df.head()
```

| | order_id | order_status | order_purchase_timestamp | order_approved_at | order_delivered_carrier_date |
|---|---|---|---|---|---|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:07:15 | 2017-10-04 19:55:00 |
| 1 | e481f51cbdc54678b7cc49136f2d6af7 | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:07:15 | 2017-10-04 19:55:00 |
| 2 | e481f51cbdc54678b7cc49136f2d6af7 | delivered | 2017-10-02 10:56:33 | 2017-10-02 11:07:15 | 2017-10-04 19:55:00 |
| 3 | 53cdb2fc8bc7dce0b6741e2150273451 | delivered | 2018-07-24 20:41:37 | 2018-07-26 03:24:27 | 2018-07-26 14:31:00 |
| 4 | 47770eb9100c2d0c44946d9cf07ec65d | delivered | 2018-08-08 08:38:49 | 2018-08-08 08:55:23 | 2018-08-08 13:50:00 |

```
df['order_id'].nunique()
```

```
99441
```

## preparing data at order level

```
df_order = df.groupby('order_id').agg({

    # customer info (same per order)
    'customer_unique_id': 'first',
    'order_status': 'first',
    'order_purchase_timestamp': 'first',
    'order_approved_at': 'first',
    'order_delivered_carrier_date': 'first',
    'order_delivered_customer_date': 'first',
    'order_estimated_delivery_date': 'first',
    'customer_city': 'first',
```

```
        'customer_state': 'first',
        'customer_zip_code_prefix': 'first',

        # review
        'review_score': 'mean',

        # payment aggregation
        'payment_value': 'sum',
        'payment_installments': 'max',
        'payment_sequential': 'max',
        'payment_type': lambda x: list(x)

    }).reset_index()
```

```
# is_emi
df_order['is_emi'] = (df_order['payment_installments'] > 1).astype(int)

# is_boleto
df_order['is_boleto'] = df_order['payment_type'].apply(
    lambda x: 1 if 'boleto' in x else 0
)

# is_multi_payment
df_order['is_multi_payment'] = df_order['payment_type'].apply(
    lambda x: 1 if len(set(x)) > 1 else 0
)

# n_vouchers
df_order['n_vouchers'] = df_order['payment_type'].apply(
    lambda x: x.count('voucher')
)

# is_late
df_order['order_delivered_customer_date'] = pd.to_datetime(df_order['order_delivered_customer_date'])
df_order['order_estimated_delivery_date'] = pd.to_datetime(df_order['order_estimated_delivery_date'])

df_order['is_late'] = (
    df_order['order_delivered_customer_date'] > df_order['order_estimated_delivery_date']
).astype(int)


# n_payments
df_order['n_payments'] = df_order['payment_sequential']

# amount
df_order['amount'] = df_order['payment_value']

# city
df_order['city'] = df_order['customer_city']

# state
df_order['state'] = df_order['customer_state']

# zip_code
df_order['zip_code'] = df_order['customer_zip_code_prefix']
```

```
df = df_order.drop(columns=[
    'customer_city',
    'customer_state',
    'customer_zip_code_prefix',
    'payment_sequential',
    'payment_value'
])
```

```
df_order = df_order.drop(columns=['payment_type'])
```

```
print(df_order.shape)
df_order.head()
```

```
(99441, 25)
```

| | order_id | customer_unique_id | order_status | order_purchase_timestamp | order_approve |
|---|---|---|---|---|---|
| 0 | 00010242fe8c5a6d1ba2dd792cb16214 | 871766c5855e863f6eccc05f988b23cb | delivered | 2017-09-13 08:59:02 | 2017-09-13 09: |
| 1 | 00018f77f2f0320c557190d7a144bdd3 | eb28e67c4c0b83846050ddfb8a35d051 | delivered | 2017-04-26 10:53:06 | 2017-04-26 11: |
| 2 | 000229ec398224ef6ca0657da4fc703e | 3818d81c6709e39d06b2738a8d3a2474 | delivered | 2018-01-14 14:33:31 | 2018-01-14 14: |
| 3 | 00024acbcdf0a6daa1e931b038114c75 | af861d436cfc08b2c2ddefd0ba074622 | delivered | 2018-08-08 10:00:35 | 2018-08-08 10: |
| 4 | 00042b26cf59d7ce69dfabb4e55b4fd9 | 64b576fb70d441e8f1b2d7d446e483c5 | delivered | 2017-02-04 13:57:51 | 2017-02-04 14: |

```
df_late = df_order[df_order['is_late'] == 1]
df_late.shape
# 1 out of 12 orders are being delivered late
```

```
(7827, 25)
```

```
city_stats = df_order.groupby('city').agg({
    'is_late': 'sum',      # late count
    'order_id': 'count'    # total orders
})
city_stats['total_orders']=city_stats['order_id']
city_stats.drop(columns=['order_id'], inplace=True)

city_stats['late_pct'] = (city_stats['is_late'] / city_stats['total_orders']) * 100
city_stats = city_stats[city_stats['total_orders'] > 40]
city_stats = city_stats[city_stats['late_pct'] > 20]

city_stats.sort_values(
    by=['total_orders', 'late_pct'],
    ascending=[False, False],
    inplace=True
)

city_stats.head(15)
```

| city | is_late | total_orders | late_pct |
|---|---|---|---|
| sao goncalo | 96 | 409 | 23.471883 |
| sao luis | 75 | 353 | 21.246459 |
| teresina | 60 | 281 | 21.352313 |
| maceio | 70 | 247 | 28.340081 |
| hortolandia | 30 | 145 | 20.689655 |
| sao joao de meriti | 27 | 134 | 20.149254 |
| cabo frio | 27 | 117 | 23.076923 |
| camacari | 13 | 61 | 21.311475 |
| itaperuna | 12 | 47 | 25.531915 |
| santarem | 15 | 44 | 34.090909 |
| juazeiro | 9 | 44 | 20.454545 |
| porto seguro | 10 | 41 | 24.390244 |

Next steps: Generate code with `city_stats` New interactive sheet

In above mentioned 10 cities, must needed to improve the shipping & delivery process

```
df_order['order_status'].value_counts()
```

| order_status | count |
|---|---|
| delivered | 96478 |
| shipped | 1107 |
| canceled | 625 |
| unavailable | 609 |
| invoiced | 314 |
| processing | 301 |
| created | 5 |
| approved | 2 |

**dtype:** int64

```
df_order.groupby('order_status')['is_boleto'].mean() * 100
# Boleto is NOT a key driver — just a minor behavioral signal
```

| order_status | is_boleto |
|---|---|
| approved | 0.000000 |
| canceled | 15.200000 |
| created | 40.000000 |
| delivered | 19.891582 |
| invoiced | 21.337580 |
| processing | 23.255814 |
| shipped | 18.879855 |
| unavailable | 24.630542 |

**dtype:** float64

## preparing data at customer level

```
# ensure datetime
df_order['order_purchase_timestamp'] = pd.to_datetime(df_order['order_purchase_timestamp'])

# snapshot date (today in dataset)
snapshot_date = df_order['order_purchase_timestamp'].max()
```

```
customer = df_order.groupby('customer_unique_id').agg(

    # 1. total orders
    total_orders=('order_id', 'count'),

    # 2. delivered
    total_delivered=('order_status', lambda x: (x == 'delivered').sum()),

    # 3. cancelled
    total_cancelled=('order_status', lambda x: (x == 'canceled').sum()),

    # 4. late & on-time
    total_late=('is_late', 'sum'),
    total_ontime=('is_late', lambda x: (x == 0).sum()),

    # 5. review avg (conditional)
    avg_review_ontime=('review_score', lambda x: x[df_order.loc[x.index, 'is_late'] == 0].mean()),
    avg_review_late=('review_score', lambda x: x[df_order.loc[x.index, 'is_late'] == 1].mean()),
    avg_review_score=('review_score','mean'),

    # 6. RFM
    recency=('order_purchase_timestamp', lambda x: (snapshot_date - x.max()).days),
    frequency=('order_id', 'count'),
    monetary=('amount', 'sum'),

    # 7. payment behavior
    avg_installments=('payment_installments', 'mean'),
    emi_ratio=('is_emi', 'mean'),
```

```
    # 8. tenure
    first_order=('order_purchase_timestamp', 'min'),
    last_order=('order_purchase_timestamp', 'max'),

    # 9. boleto
    boleto_orders=('is_boleto', 'sum'),

    # 10. payment modes
    multi_payment_orders=('is_multi_payment', 'sum'),
    single_payment_orders=('is_multi_payment', lambda x: (x == 0).sum()),

    # 11. vouchers
    voucher_orders=('n_vouchers', lambda x: (x > 0).sum()),
    total_vouchers=('n_vouchers', 'sum'),
    voucher_amount=('amount', lambda x: x[df_order.loc[x.index, 'n_vouchers'] > 0].sum())

).reset_index()
```

```
import numpy as np

# -------------------------
# TENURE
# -------------------------
customer['tenure_days'] = (customer['last_order'] - customer['first_order']).dt.days

# -------------------------
# CHURN (data-driven)
# -------------------------
threshold = customer['recency'].quantile(0.75)
customer['churn'] = (customer['recency'] > threshold).astype(int)

# -------------------------
# SAFE DIVISION HELPER
# -------------------------
def safe_divide(a, b):
    return np.where(b == 0, 0, a / b)

# -------------------------
# RATIOS
# -------------------------
customer['late_ratio'] = safe_divide(customer['total_late'], customer['total_orders'])
customer['cancel_ratio'] = safe_divide(customer['total_cancelled'], customer['total_orders'])
customer['boleto_ratio'] = safe_divide(customer['boleto_orders'], customer['total_orders'])
customer['multi_payment_ratio'] = safe_divide(customer['multi_payment_orders'], customer['total_orders'])

# -------------------------
# VALUE FEATURES
# -------------------------
customer['avg_order_value'] = safe_divide(customer['monetary'], customer['total_orders'])

# active days
customer['active_days'] = customer['tenure_days'] + 1 - customer['recency']
customer['active_days'] = customer['active_days'].clip(lower=0)

# -------------------------
# FREQUENCY (TIME-BASED)
# -------------------------
customer['days/orders'] = safe_divide(customer['tenure_days'], customer['total_orders'])

# -------------------------
# REVIEW NaNs
# -------------------------
customer['avg_review_late'] = customer['avg_review_late'].fillna(0)
customer['avg_review_ontime'] = customer['avg_review_ontime'].fillna(0)

# -------------------------
# HIGH-VALUE FEATURES
# -------------------------

# 1. Delivery issue score
customer['delivery_issue_score'] = customer['late_ratio'] + customer['cancel_ratio']

# 2. Repeat customer
customer['is_repeat_customer'] = (customer['total_orders'] > 1).astype(int)


# -------------------------
# ORDER VALUE CONSISTENCY
# -------------------------
order_std = df_order.groupby('customer_unique_id')['amount'].std().reset_index()
order_std.columns = ['customer_unique_id', 'order_value_std']
```

```
customer = customer.merge(order_std, on='customer_unique_id', how='left')
customer['order_value_std'] = customer['order_value_std'].fillna(0)
```

## Customer Segmentation (RFM) & Analysis

```
single_order_customers = customer[customer['total_orders']==1]
single_order_customers['monetary'].quantile(0.9)
```

```
np.float64(309.54599999999994)
```

1. most of the customers have been placed only 1 order, so we cant decide their nature
2. so, we are going to promote more to the one's whose order amount > 90% of the people
3. reason: they already made a move by buying an expensive item, so we can target them by promoting more & offering vouchers

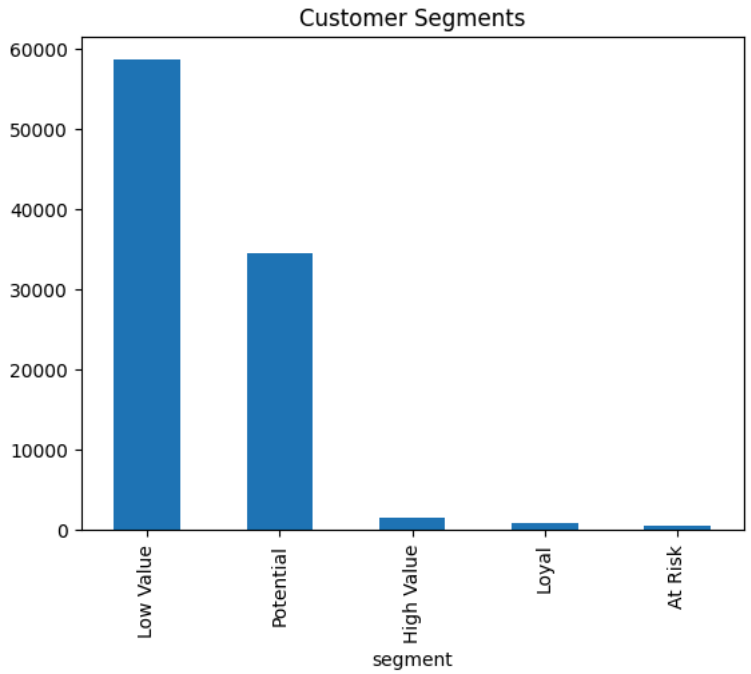
```
# creating RFM scores
def f_score(x):
  # wrote a sep function coz the qcut fails, due to most of the customers had the frequency=1, fails to make bins due to da
  # [1,1,1,17] - fails to split into 4 bins

   if x == 1:
       return 1
   elif x <= 3:
       return 2
   elif x <= 5:
       return 3
   else:
       return 4


 # R → lower is better → reverse scoring
customer['r_score'] = pd.qcut(customer['recency'], 4, labels=[4,3,2,1]).astype(int)

# F → higher is better
customer['f_score'] = customer['frequency'].apply(f_score)

# M → higher is better
customer['m_score'] = pd.qcut(customer['monetary'], 4, labels=[1,2,3,4]).astype(int)
```

```
# combining rfm
customer['rfm_score'] = (
    0.5*customer['r_score'] +
    0.1*customer['f_score'] +
    0.4*customer['m_score']
)
```

```
monetary_30 = customer['monetary'].quantile(0.3)
recency_med = customer['recency'].median()

def segment(row):
    r, f, m = row['r_score'], row['f_score'], row['m_score']

    # 🔴 LOW VALUE (override)
    if (
        (row['monetary'] < monetary_30) and
        (row['total_orders'] == 1) and
        (row['recency'] > recency_med)
    ):
        return 'Low Value'

    # 🟢 TRUE HIGH VALUE (must be repeat + high spend + recent)
    if (row['total_orders'] > 1) and (r >= 3) and (m >= 3):
        return 'High Value'

    # 🔵 LOYAL (repeat customers)
    elif row['total_orders'] > 1 and r >= 2:
        return 'Loyal'

    # 🟠 AT RISK (repeat but inactive)
    elif row['total_orders'] > 1 and r <= 2:
        return 'At Risk'

    # 🟡 POTENTIAL (recent but single purchase)
    elif r >= 3 and m >= 2:
        return 'Potential'
```

```
    # 🟣 fallback
    else:
        return 'Low Value'


customer['segment'] = customer.apply(segment, axis=1)
```

```
import matplotlib.pyplot as plt

# segment counts
segment_counts = customer['segment'].value_counts()

segment_counts.plot(kind='bar', title="Customer Segments")
plt.show()
```




```
segment_counts.head()
```

| segment | count |
|---|---|
| **Low Value** | 58618 |
| **Potential** | 34481 |
| **High Value** | 1485 |
| **Loyal** | 897 |
| **At Risk** | 615 |

**dtype:** int64

**Low Value + Potential = ~80%+**

**Meaning:** Company is good at acquiring users BUT bad at retaining them

**Focus shift**: Acquisition → Retention

1. **LOW VALUE: 1-time buyers, low spend, in-active**

- Largest group, Reduce cost, don't chase them
- Don't do: heavy discounts & paid ads targetting on them
- Use automated emails, low cost nudges

2. **POTENTIAL: recent user's, not loyal yet!**

- Retention needs to be stronger !
- Strongly Offer: first-repeat incentives, personalized offers, reminders

3. **HIGH VALUE: repeat,high-spent**

- **Protect them at all cost**

- VIP treatment, priority delivery, exclusive deals

4. **LOYAL: repeat buyers, moderate spend**

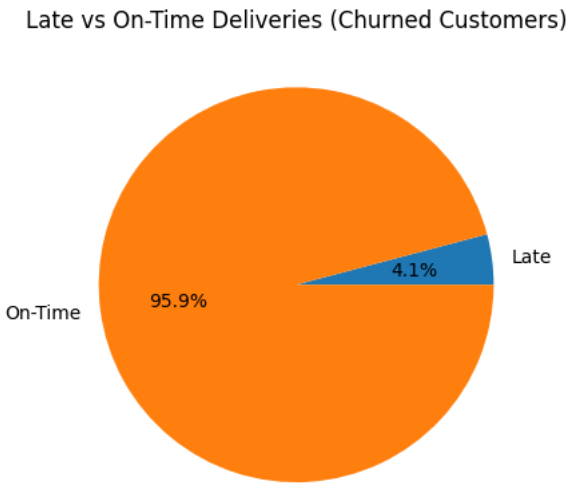
- Increase basket size
- cross-sell, bundles, loyalty points

5. **AT RISK: used to buy, now in-active**

- Win them back quickly
- reactivation campaigns, discounts, reminders

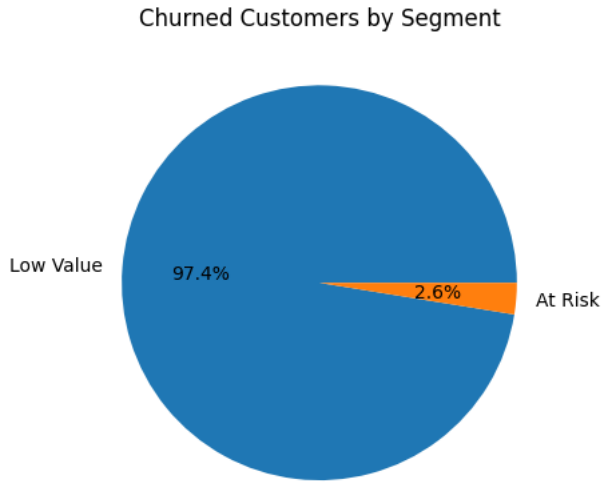
## checking the late delivery impact

```
import matplotlib.pyplot as plt

# filter churned users
churned = customer[customer['churn'] == 1]

# aggregate late vs on-time
late_data = {
    'Late': churned['total_late'].sum(),
    'On-Time': churned['total_ontime'].sum()
}

# plot
plt.figure()
plt.pie(late_data.values(), labels=late_data.keys(), autopct='%1.1f%%')
plt.title("Late vs On-Time Deliveries (Churned Customers)")
plt.show()
```

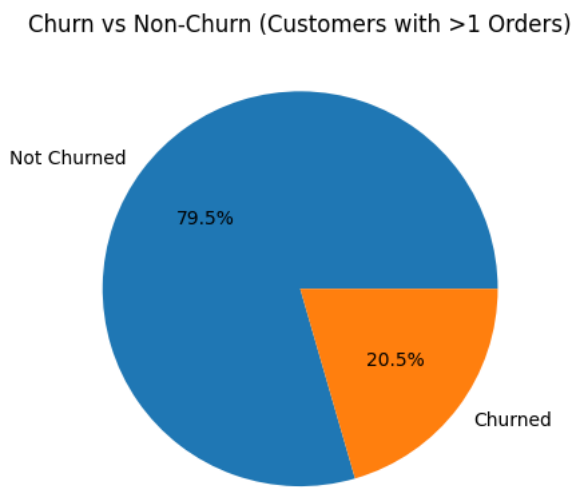



for churning the late delivery is not an issue, poor customer retention is the issue

```
churn_by_segment = customer[customer['churn'] == 1]['segment'].value_counts()

plt.figure()
plt.pie(churn_by_segment, labels=churn_by_segment.index, autopct='%1.1f%%')
plt.title("Churned Customers by Segment")
plt.show()
```




```
import matplotlib.pyplot as plt

# filter customers with >1 orders
df = customer[customer['total_orders'] > 1]

# count churn vs non-churn
churn_counts = df['churn'].value_counts()

# labels
labels = ['Not Churned', 'Churned']

# plot
plt.figure()
plt.pie(churn_counts, labels=labels, autopct='%1.1f%%')
plt.title("Churn vs Non-Churn (Customers with >1 Orders)")
plt.show()
```




```
customer[customer['churn'] == 1].shape
```

```
(23954, 39)
```

0-90 days after the client aquisition is super important for the retention: with in that period only so many have churned

## MODEL & STRATEGY FOR CUSTOMERS WITH TOTAL ORDERS > 1

### feature engineering for model building

```
customer.head()
```

| | customer_unique_id | total_orders | total_delivered | total_cancelled | total_late | total_ontime | avg_review_ont |
|---|---|---|---|---|---|---|---|
| **0** | 0000366f3b9a7992bf8c76cfdf3221e2 | 1 | 1 | 0 | 0 | 1 | |
| **1** | 0000b849f77a49e4a4ce2b2a4ca5be3f | 1 | 1 | 0 | 0 | 1 | |
| **2** | 0000f46a3911fa3c0805444483337064 | 1 | 1 | 0 | 0 | 1 | |
| **3** | 0000f6ccb0745a6a4b88665a16c9f078 | 1 | 1 | 0 | 0 | 1 | |
| **4** | 0004aac84e0df4da2b147fca70cf8255 | 1 | 1 | 0 | 0 | 1 | |

Goal: predict the churn before it can happen, not after. so the dataset we are going to build is going to look like:

| customer | date | orders_till_now | avg_value | churn_next_60_days |
|---|---|---|---|---|
| A | Jan | 2 | 100 | 0 |
| A | Feb | 3 | 120 | 1 |

```
import pandas as pd

# -------------------------
# 1. PREPARE DATA
# -------------------------
df_order['order_purchase_timestamp'] = pd.to_datetime(df_order['order_purchase_timestamp'])

# latest date in dataset
today = df_order['order_purchase_timestamp'].max()

# -------------------------
# 2. AGGREGATE CUSTOMER LEVEL
# -------------------------

# total orders
orders = df_order.groupby('customer_unique_id')['order_id'].count().reset_index()
orders.columns = ['customer_unique_id', 'total_orders']

# monetary
monetary = df_order.groupby('customer_unique_id')['amount'].sum().reset_index()
monetary.columns = ['customer_unique_id', 'monetary']

# avg order value
avg_value = df_order.groupby('customer_unique_id')['amount'].mean().reset_index()
avg_value.columns = ['customer_unique_id', 'avg_order_value']

# late ratio
late = df_order.groupby('customer_unique_id')['is_late'].mean().reset_index()
late.columns = ['customer_unique_id', 'late_ratio']

# cancel count
cancel = (
    (df_order['order_status'] == 'canceled')
    .groupby(df_order['customer_unique_id'])
    .sum()
    .reset_index(name='cancel_count')
)

# total late count
late_count = df_order.groupby('customer_unique_id')['is_late'].sum().reset_index()
late_count.columns = ['customer_unique_id', 'late_count']

# avg time between orders
df_order = df_order.sort_values(['customer_unique_id', 'order_purchase_timestamp'])

df_order['prev_order'] = df_order.groupby('customer_unique_id')['order_purchase_timestamp'].shift(1)
df_order['gap'] = (df_order['order_purchase_timestamp'] - df_order['prev_order']).dt.days

avg_gap = df_order.groupby('customer_unique_id')['gap'].mean().reset_index()
avg_gap.columns = ['customer_unique_id', 'avg_time_between_orders']
avg_gap['avg_time_between_orders'] = avg_gap['avg_time_between_orders'].fillna(0)

# last order (ONLY for label, will drop later)
last_order = df_order.groupby('customer_unique_id')['order_purchase_timestamp'].max().reset_index()
last_order.columns = ['customer_unique_id', 'last_order']

# recency (ONLY for label)
last_order['recency'] = (today - last_order['last_order']).dt.days
```

```
# -------------------------
# 3. CHURN LABEL (YOUR LOGIC)
# -------------------------
churn_threshold = last_order['recency'].quantile(0.75) # YOU DECIDED THIS

last_order['churn'] = (last_order['recency'] > churn_threshold).astype(int)

churn_df = last_order[['customer_unique_id', 'churn']]

# -------------------------
# EXTRA FEATURES
# -------------------------

# avg review score
review = df_order.groupby('customer_unique_id')['review_score'].mean().reset_index()
review.columns = ['customer_unique_id', 'avg_review_score']

# order value std
order_std = df_order.groupby('customer_unique_id')['amount'].std().reset_index()
order_std.columns = ['customer_unique_id', 'order_value_std']
order_std['order_value_std'] = order_std['order_value_std'].fillna(0)

# -------------------------
# 4. MERGE EVERYTHING
# -------------------------
df_model = churn_df.copy()

for df in [
    orders, monetary, avg_value, late,
    cancel, late_count, avg_gap,
    review, order_std
]:
    df_model = df_model.merge(df, on='customer_unique_id', how='left')

df_model = df_model.fillna(0)

# -------------------------
# 5. REMOVE 1-ORDER USERS
# -------------------------
df_model = df_model[df_model['total_orders'] > 1]


# ----------------------------------------------------
# 6. EXTRA FEATURES FOR BETTER SIGNAL TOWARDS CHURN
# ----------------------------------------------------

df_model['recency_ratio'] = df_model['avg_time_between_orders'] / (df_model['avg_time_between_orders'].mean()) #recency rat

df_model['order_consistency'] = 1 / (df_model['order_value_std'] + 1) # order consistency

df_model['experience_score'] = (
    df_model['avg_review_score'] -
    (df_model['late_ratio'] * 2) -              # user-experience score
    (df_model['cancel_count'] * 0.5)
)


df_model['bad_experience'] = df_model['late_count'] + df_model['cancel_count']    # negative experience count


df_model['value_density'] = df_model['monetary'] / (df_model['total_orders'] + 1) # value density: helps distinguish high v

df_model['engagement_score'] = df_model['total_orders'] / (df_model['avg_time_between_orders'] + 1)  # engagement strengtgh

df_model['risk_score'] = (
    df_model['late_ratio'] * df_model['cancel_count']     # risk interaction
)

df_model['review_std_proxy'] = abs(df_model['avg_review_score'] - 3)   # review instability: extreme ratings--> unstable us


# already have gap — reuse logic
gap_trend = df_order.sort_values(['customer_unique_id','order_purchase_timestamp'])

gap_trend['prev_gap'] = gap_trend.groupby('customer_unique_id')['gap'].shift(1)

gap_trend['gap_change'] = gap_trend['gap'] - gap_trend['prev_gap']

trend = gap_trend.groupby('customer_unique_id')['gap_change'].mean().reset_index()
trend.columns = ['customer_unique_id', 'gap_trend']
```

```
df_model = df_model.merge(trend, on='customer_unique_id', how='left')
df_model['gap_trend'] = df_model['gap_trend'].fillna(0)


# -------------------------
# 7. CHECK
# -------------------------
print("Churn Distribution:\n")
print(df_model['churn'].value_counts(normalize=True))
```

```
Churn Distribution:

churn
0    0.794795
1    0.205205
Name: proportion, dtype: float64
```

```
# ensure datetime
df_order['order_purchase_timestamp'] = pd.to_datetime(df_order['order_purchase_timestamp'])

# quantiles of order dates (overall)
quantiles = df_order['order_purchase_timestamp'].quantile(
    [0.0,0.1, 0.25, 0.5, 0.7, 0.75, 0.9,1.0]
)

print(quantiles)
```

```
0.00   2016-09-04 21:15:19
0.10   2017-05-17 15:42:02
0.25   2017-09-12 14:46:19
0.50   2018-01-18 23:04:36
0.70   2018-04-13 21:53:34
0.75   2018-05-04 15:42:16
0.90   2018-07-18 10:33:46
1.00   2018-10-17 17:30:18
Name: order_purchase_timestamp, dtype: datetime64[ns]
```

## model training

```
import numpy as np

# nulls
print("Missing values:\n", df_model.isnull().sum())

# inf check
print("\nInf values:\n", np.isinf(df_model.select_dtypes(include=[float, int])).sum())

# class balance
print("\nClass distribution:\n", df_model['churn'].value_counts(normalize=True))
```

```
Missing values:
 customer_unique_id         0
churn                      0
total_orders               0
monetary                   0
avg_order_value            0
late_ratio                 0
cancel_count               0
late_count                 0
avg_time_between_orders    0
avg_review_score           0
order_value_std            0
recency_ratio              0
order_consistency          0
experience_score           0
bad_experience             0
value_density              0
engagement_score           0
risk_score                 0
review_std_proxy           0
gap_trend                  0
dtype: int64

Inf values:
 churn                      0
total_orders               0
monetary                   0
avg_order_value            0
late_ratio                 0
cancel_count               0
late_count                 0
avg_time_between_orders    0
avg_review_score           0
```

```
order_value_std            0
recency_ratio              0
order_consistency          0
experience_score           0
bad_experience             0
value_density              0
engagement_score           0
risk_score                 0
review_std_proxy           0
gap_trend                  0
dtype: int64

Class distribution:
 churn
0    0.794795
1    0.205205
Name: proportion, dtype: float64
```

```
# -------------------------
# SELECT FEATURES FOR MODEL
# -------------------------

features = [
    'total_orders',
    'monetary',
    'avg_order_value',
    'late_ratio',
    'cancel_count',
    'late_count',
    'avg_time_between_orders',
    'avg_review_score',
    'order_value_std',

    # NEW
    'recency_ratio',
    'order_consistency',
    'experience_score',
    'bad_experience',
    'value_density',
    'engagement_score',
    'risk_score',
    'review_std_proxy',
    'gap_trend'
]

X = df_model[features].copy()
y = df_model['churn'].copy()
```

```
# train-test split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

```
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
```

```
# model + randomized searchCV
import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from scipy.stats import uniform, randint

xgb_model = xgb.XGBClassifier(
    objective='binary:logistic',
    eval_metric='aucpr',   # PR-focused
    tree_method='hist',
    scale_pos_weight=scale_pos_weight,
    random_state=42
)

param_dist = {
    # model size
    'n_estimators': randint(300, 1000),

    # complexity control
    'max_depth': randint(3, 8),
    'min_child_weight': randint(1, 10),
```

```
    # learning dynamics (avoid local minima)
    'learning_rate': uniform(0.01, 0.15),

    # randomness (VERY IMPORTANT)
    'subsample': uniform(0.6, 0.4),
    'colsample_bytree': uniform(0.6, 0.4),

    # regularization (overfit control)
    'gamma': uniform(0, 5),
    'reg_alpha': uniform(0, 5),     # L1
    'reg_lambda': uniform(1, 10)    # L2
}

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=param_dist,
    n_iter=25,
    scoring='average_precision',   # PR-AUC optimization
    cv=cv,
    verbose=1,
    n_jobs=-1,
    random_state=42
)
```

```
random_search.fit(X_train, y_train)

best_params = random_search.best_params_
print("Best params:\n", best_params)
```

```
Fitting 3 folds for each of 25 candidates, totalling 75 fits
Best params:
 {'colsample_bytree': np.float64(0.9261845713819337), 'gamma': np.float64(3.534286719238086), 'learning_rate': np.float64(0.
```

```
# fitting final model with early stopping
final_model = xgb.XGBClassifier(
    **best_params,
    objective='binary:logistic',
    eval_metric='aucpr',
    scale_pos_weight=scale_pos_weight,
    tree_method='hist',
    random_state=42,
    early_stopping_rounds=5   # correct placement for your version
)

final_model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=False
)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=np.float64(0.9261845713819337), device=None,
              early_stopping_rounds=5, enable_categorical=False,
              eval_metric='aucpr', feature_types=None, feature_weights=None,
              gamma=np.float64(3.534286719238086), grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=np.float64(0.11935107520614809), max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=3, max_leaves=None,
              min_child_weight=5, missing=nan, monotone_constraints=None,
```

RECALL should be more than PRECISION

- we dont want to miss a customer, we try max to retain them
- the aquisition cost of a customer is more than spending on email campaigns / offering a discount voucher
- if we want to offer a 50 dollar's credit/ free month of service, this is another scenario
- for now most of the customers are chrning, provinding the 50$ credit/ free month of service is practically imposssible
- so we focus on identifying the customer who is at risk of getting churn & our attenmpts should be towards retaining that customer

```
from sklearn.metrics import (
    precision_score, recall_score,
    roc_auc_score, average_precision_score,
    classification_report,accuracy_score
```

```
)

y_prob = final_model.predict_proba(X_test)[:, 1]
y_pred = (y_prob >= 0.5).astype(int)

y_test_prob = final_model.predict_proba(X_train)[:, 1]
y_train_pred= (y_test_prob >= 0.5).astype(int)

print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_prob))
print("PR-AUC:", average_precision_score(y_test, y_prob))
print("Train Accuracy:",accuracy_score(y_train,y_train_pred))
print("Test Accuracy:",accuracy_score(y_test,y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Precision: 0.2881844380403458
Recall: 0.8130081300813008
ROC-AUC: 0.7112542823541443
PR-AUC: 0.33510861678431236
Train Accuracy: 0.5803087192323738
Test Accuracy: 0.55

Classification Report:

              precision    recall  f1-score   support

           0       0.91      0.48      0.63       477
           1       0.29      0.81      0.43       123

    accuracy                           0.55       600
   macro avg       0.60      0.65      0.53       600
weighted avg       0.78      0.55      0.59       600
```

```
# applying custom threshold
y_pred_opt = (y_prob >= 0.45).astype(int)

print("Optimized Precision:", precision_score(y_test, y_pred_opt))
print("Optimized Recall:", recall_score(y_test, y_pred_opt))
```

```
Optimized Precision: 0.2576112412177986
Optimized Recall: 0.8943089430894309
```

## Saving the best model

```
import pickle

with open('final_model.pkl', 'wb') as f:
    pickle.dump(final_model, f)
```

```
# RIGHT AFTER YOU DEFINED X
feature_names = X.columns.tolist()

import pickle
with open('feature_names.pkl', 'wb') as f:
    pickle.dump(feature_names, f)
```

## Churn Risk + Customer Value → Action Strategy

```
# Define VALUE SEGMENTS (simple & effective)
def value_segment(row):
    if row['monetary'] > high_value_thresh and row['total_orders'] > 3:
        return 'High'
    elif row['monetary'] > mid_value_thresh:
        return 'Medium'
    else:
        return 'Low'
high_value_thresh = customer['monetary'].quantile(0.75)
mid_value_thresh  = customer['monetary'].quantile(0.40)
```

```
# Define CHURN RISK TIERS (based on probability)
def churn_risk(p):
    if p >= 0.5:
        return 'High Risk'
    elif p >= 0.3:
```

```
        return 'Medium Risk'
    else:
        return 'Low Risk'
```

```
# ACTION STRATEGY MATRIX (THIS IS THE CORE)
```

| Value ↓ / Risk → | High Risk | Medium Risk | Low Risk |
|---|---|---|---|
| 🟢 High Value | 💥 Strong retention | 🎯 Engage | 😊 Light touch |
| 🟡 Medium Value | 🎁 Small incentive | 📩 Reminder | 🙂 Passive |
| 🔴 Low Value | 📩 Reminder only | 👀 Ignore/light | ❌ Ignore |

```
# Convert to actual actions
def action_plan(row):

    if row['risk'] == 'High Risk':

        if row['value'] == 'High':
            return 'Send voucher + strong reminder + priority campaign'

        elif row['value'] == 'Medium':
            return 'Send reminder + 5% discount'

        else:
            return 'Send caring reminder only'

    elif row['risk'] == 'Medium Risk':

        if row['value'] == 'High':
            return 'Send reminder + personalized recommendation'

        elif row['value'] == 'Medium':
            return 'Send reminder'

        else:
            return 'Minimal engagement'

    else:
        return 'No action'
```

```
# Apply on your data
#customer['value'] = customer.apply(value_segment, axis=1)
#customer['risk'] = y_proba.apply(churn_risk)

#customer['action'] = customer.apply(action_plan, axis=1)
```

## STRATEGY FOR CUSTOMERS WITH TOTAL ORDERS = 1

```
df_1order = df_order.groupby('customer_unique_id').filter(lambda x: len(x) == 1).copy()

df_1order = df_1order.drop_duplicates('customer_unique_id')

high_value_thresh = df_1order['amount'].quantile(0.75)

# scoring
def repeat_score(row):
    score = 0

    if row['amount'] > high_value_thresh:
        score += 2
    if row['review_score'] >= 4:
        score += 2
    if row['is_late'] == 0:
        score += 1
    if row['is_boleto'] == 0:
        score += 1

    return score

df_1order['score'] = df_1order.apply(repeat_score, axis=1)

# bucket
df_1order['bucket'] = df_1order['score'].apply(
    lambda x: 'Potential' if x >= 3 else 'Low Intent'
)

# time feature
today = df_order['order_purchase_timestamp'].max()
```

```
df_1order['days_since_order'] = (
    today - df_1order['order_purchase_timestamp']
).dt.days
```

```
# action engine
def action_engine(row):

    days = row['days_since_order']
    bucket = row['bucket']

    # -------------------------
    # POTENTIAL USERS
    # -------------------------
    if bucket == 'Potential':

        if days <= 3:
            return ('No Action', 'Wait')

        elif days <= 7:
            return ('Reminder + Recommendations',
                    'You might like these based on your purchase')

        elif days <= 14:
            return ('5-10% Discount',
                    'Exclusive offer just for you!')

        else:
            return ('Urgency Campaign',
                    'Last chance! Offer expiring soon')

    # -------------------------
    # LOW INTENT USERS
    # -------------------------
    else:

        if days <= 5:
            return ('No Action', 'Wait')

        elif days <= 10:
            return ('Soft Reminder',
                    'Check out trending products')

        elif days <= 20:
            return ('Generic Nudge',
                    'We miss you!')

        else:
            return ('Drop User',
                    'Stop spending resources')
```

```
# applying the system
df_1order[['action', 'message']] = df_1order.apply(
    lambda row: pd.Series(action_engine(row)),
    axis=1
)
```

```
# final output
df_1order[[
    'customer_unique_id',
    'amount',
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.